REDES · UNIVERSIDAD DEL VALLE DE GUATEMALA

Laboratorio IoT con Apache Kafka

Estación meteorológica simulada · Productor / Consumidor · Payload de 3 bytes

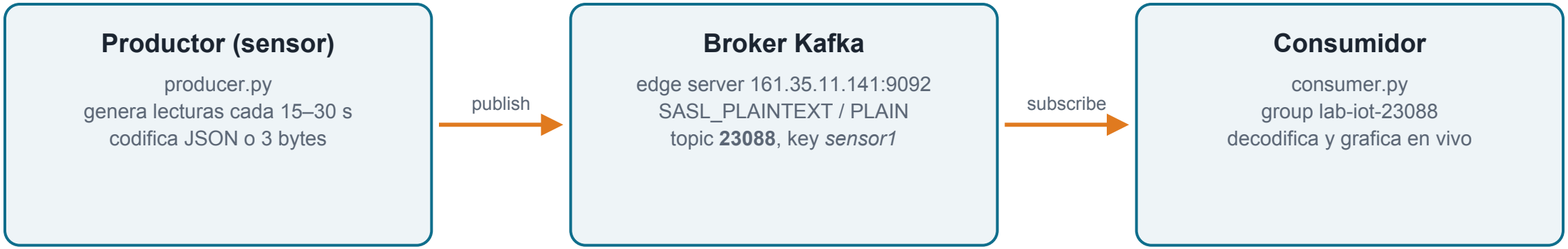
Carné 23088 · topic Kafka: 23088

Noviembre 2026

VISIÓN GENERAL

Objetivo y arquitectura

Simular una estación meteorológica IoT que publica lecturas de **temperatura**, **humedad** y **dirección del viento** a un broker Apache Kafka (edge server), y un consumidor que las recibe y grafica en tiempo real. Finalmente se restringe el payload a **3 bytes**, como en redes LPWAN (LoRa / Sigfox).



- **common.py**: configuración compartida (broker, topic, credenciales) y funciones de codificación JSON / compacta.
- **test_codec.py**: prueba de ida y vuelta (encode → decode) del formato de 3 bytes.

SECCIÓN 3.1

Fase 1 · Simulación de los sensores

- Temperatura y humedad siguen una **distribución normal** con media 55 y desviación estándar 15 (varianza 225), acotadas a sus rangos válidos.
- **Temperatura**: float de 0 a 110 °C con 2 decimales.
- **Humedad**: entero de 0 a 100 %.
- **Dirección del viento**: una de 8 posiciones {N, NO, O, SO, S, SE, E, NE}, elegida uniformemente.
- La lectura se representa como diccionario y se serializa a JSON (UTF-8) para enviarse.

```
MEDIA, SIGMA = 55.0, 15.0

def generar_data() -> dict:
    temp = min(110.0, max(0.0,
                random.gauss(MEDIA, SIGMA)))
    hum = int(round(min(100, max(0,
                random.gauss(MEDIA, SIGMA)))))
    return {
        "temperatura": round(temp, 2),
        "humedad": hum,
        "direccion_viento": random.choice(WINDS),
    }
```

```
# Ejemplo de lectura serializada (63 bytes)
{"temperatura": 55.37, "humedad": 62,
 "direccion_viento": "SE"}
```

SECCIÓN 3.2

Fase 2 · Envío de datos al Edge Server (Kafka)

- Se usa **kafka-python** (`KafkaProducer`) contra el broker del curso con autenticación **SASL/PLAIN**.
- Cada estudiante publica en su propio topic: el carné **23088**.
- Todos los mensajes llevan la key `sensor1`, por lo que caen en la misma partición y conservan el orden.
- `send(...).get(timeout=15)` espera el ACK del broker: si el envío falla se ve de inmediato.
- Intervalo aleatorio de **15 a 30 s** entre lecturas; la bandera `--fast` lo baja a 2 s para pruebas.

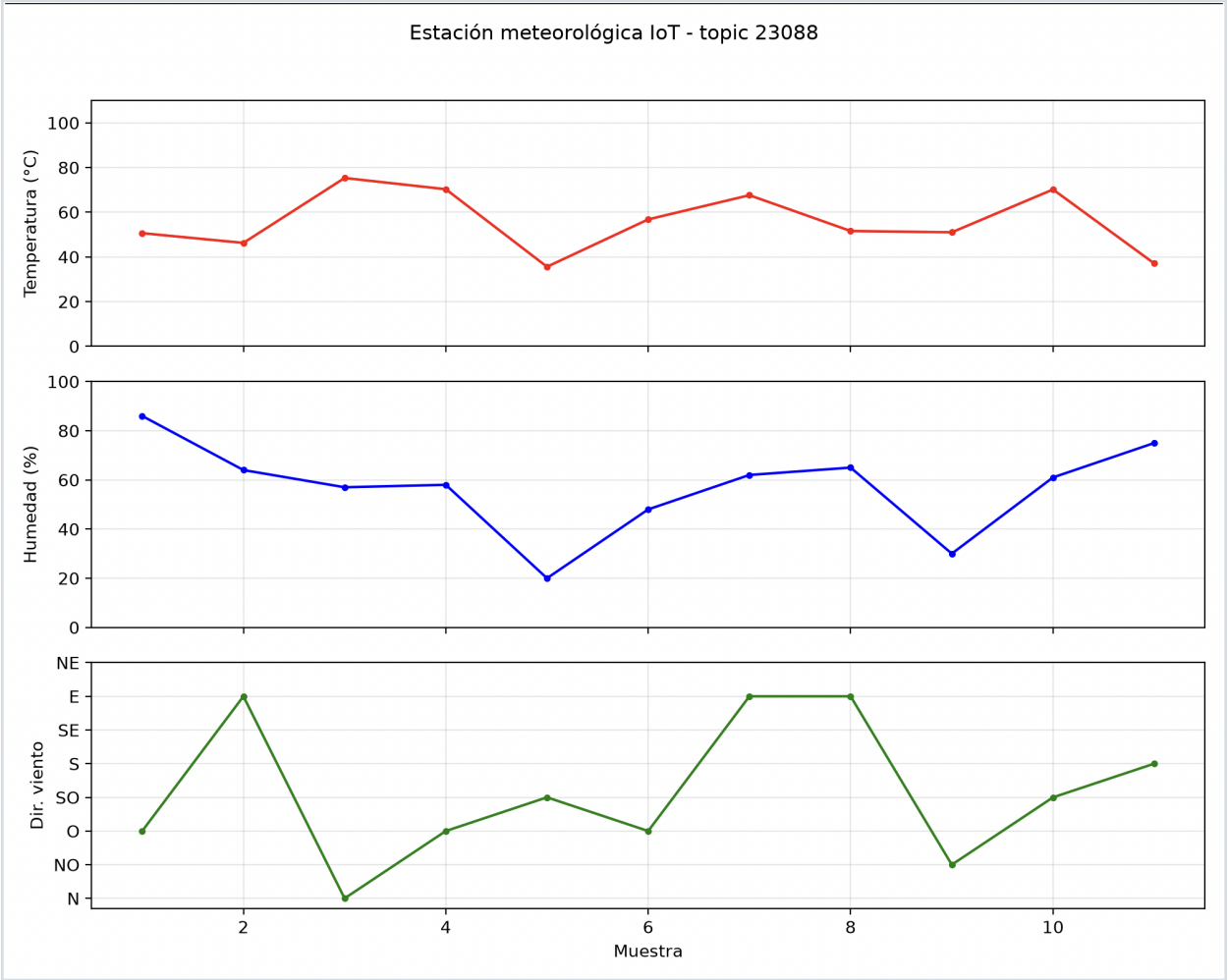
```
producer = KafkaProducer(**KAFKA_CFG)
while True:
    data  = generar_data()
    value = encode(data) if args.compact \
            else encode_json(data)
    producer.send(TOPIC, key=b"sensor1",
                  value=value).get(timeout=15)
    print(f"enviado {data} -> {value!r}")
    time.sleep(2 if args.fast
               else random.uniform(15, 30))
```

```
$ python3 producer.py           # JSON
$ python3 producer.py --fast    # cada 2 s
$ python3 producer.py --compact  # 3 bytes
```

SECCIÓN 3.3

Fase 3 · Consumir y desplegar los datos

- `KafkaConsumer` suscrito al topic 23088 con el grupo `lab-iot-23088`.
- Cada mensaje se decodifica y se agrega a las listas *all_temp*, *all_hume*, *all_wind*.
- Gráfica en vivo con matplotlib (modo interactivo): tres subgráficas que se redibujan con cada mensaje.
- La gráfica se guarda en PNG tras cada mensaje como evidencia.
- Primera corrida: productor en modo JSON con el intervalo normal de 15–30 s (11 muestras).

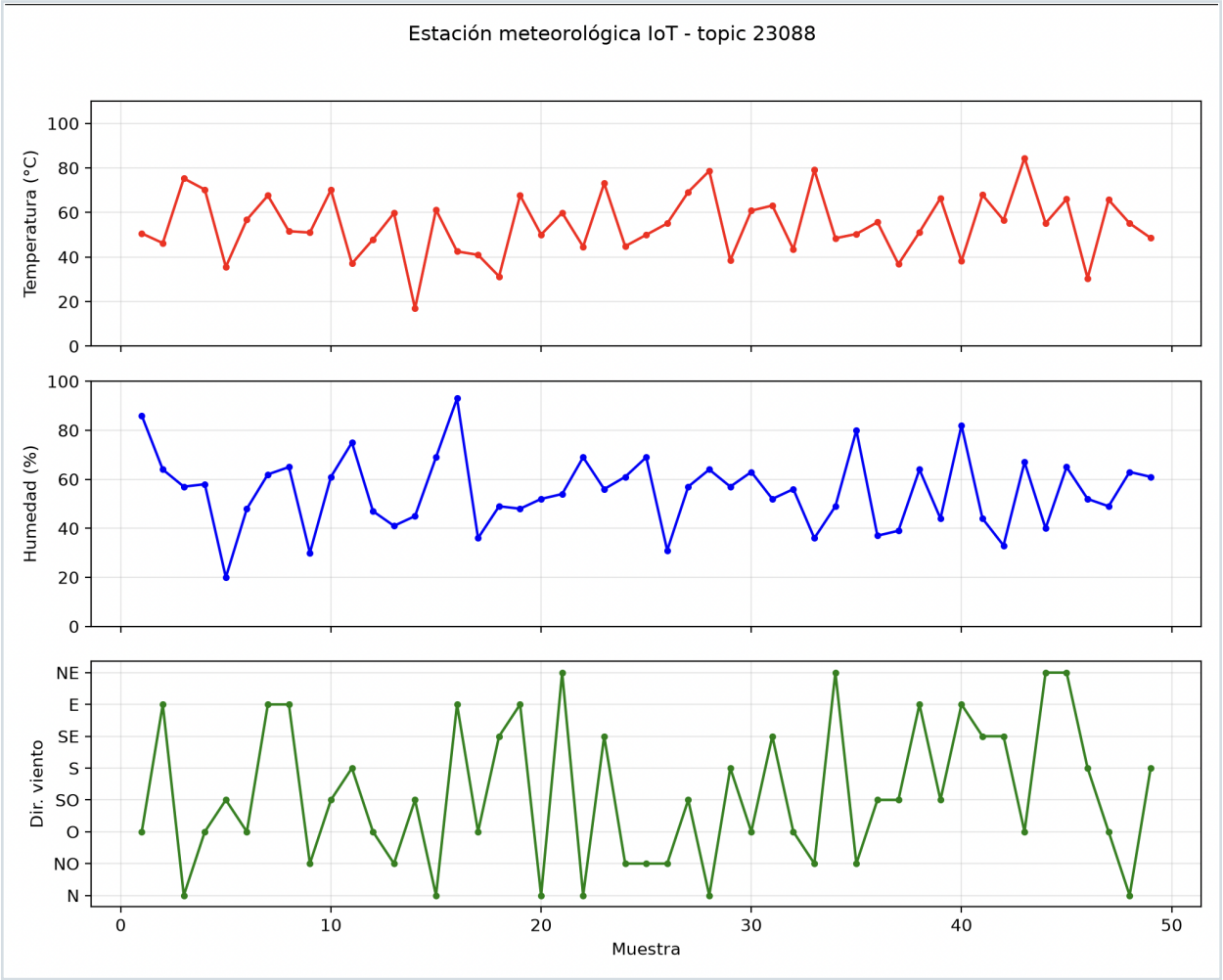


Evidencia: consumidor graficando datos JSON (intervalo de 15–30 s)

SECCIÓN 3.3

Evidencia · Prueba prolongada (modo --fast)

- Productor con `--fast` (una lectura cada 2 s) para acumular muchas muestras.
- 49 muestras recibidas sin pérdidas y en orden.
- La temperatura y la humedad oscilan alrededor de 55, como se espera de N(55, 15^2); casi todos los valores caen entre 25 y 85 (± 2σ).
- La dirección del viento recorre las 8 posiciones de forma uniforme.



Evidencia: 49 lecturas consecutivas recibidas por el consumidor

SECCIÓN 3.4

Fase 4 · Payload restringido a 3 bytes

Redes como LoRa o Sigfox limitan el tamaño de cada mensaje, así que la lectura se empaqueta en **24 bits** en lugar de enviarse como JSON:




```
t = round(temperatura * 100)       # 55.37 -> 5537
word = (t << 10) | (humedad << 3) | idx_viento
payload = word.to_bytes(3, "big")
```

```
word = int.from_bytes(raw, "big")
temperatura = ((word >> 10) & 0x3FFF) / 100
humedad     =  (word >> 3)  & 0x7F
viento      = WINDS[word & 0x7]
```

El consumidor usa `decode_auto`: si el mensaje mide 3 bytes lo decodifica como compacto; si no, como JSON. Así un mismo consumidor acepta ambos formatos.

SECCIÓN 3.4

Fase 4 · Ejemplo de codificación y verificación

```
>>> d = {"temperatura": 55.37, "humedad": 62, "direccion_viento": "SE"}
>>> encode_json(d)     # 63 bytes
b'{"temperatura": 55.37, "humedad": 62, "direccion_viento": "SE"}'
>>> encode(d).hex()    # 3 bytes
'5685f5'
>>> format(int.from_bytes(encode(d), "big"), "024b")
'01010110100001 0111110 101'
   temp = 5537    hum = 62  SE = 5
>>> decode(encode(d))
{'temperatura': 55.37, 'humedad': 62, 'direccion_viento': 'SE'}
```

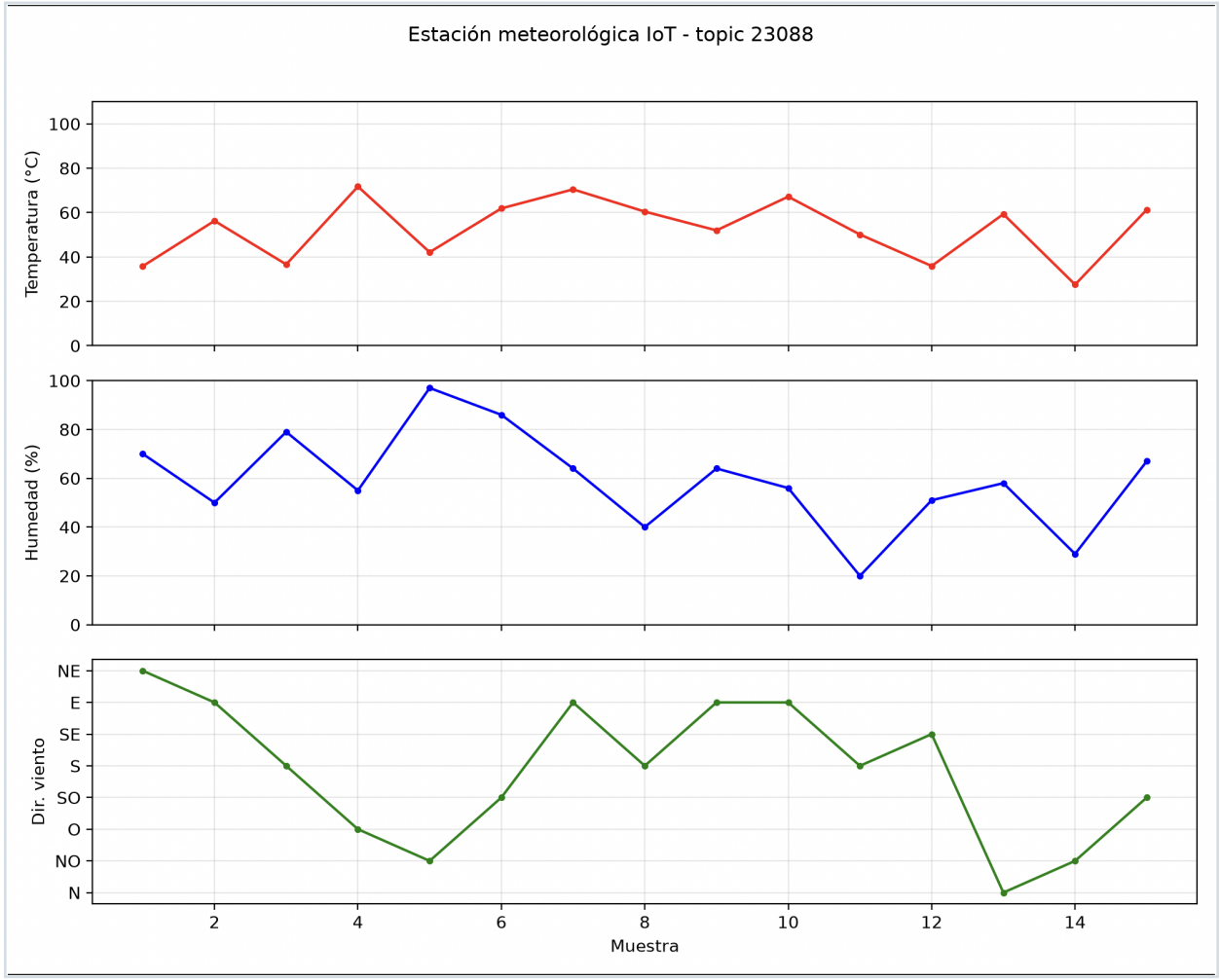
```
$ python3 test_codec.py
{'temperatura': 110.0, 'humedad': 100, 'direccion_viento': 'NE'} abe327 {...mismo valor...}
{'temperatura': 0.0, 'humedad': 0, 'direccion_viento': 'N'} 000000 {...mismo valor...}
OK: 100000 round-trips exactos, siempre 3 bytes
```

Reducción: 63 bytes → 3 bytes (≈ 95 % menos). La prueba recorre 100 000 lecturas aleatorias y los extremos del rango sin perder precisión.

SECCIÓN 3.4

Evidencia · Consumidor recibiendo payloads de 3 bytes

- Productor ejecutado con `--compact`: cada mensaje en Kafka mide 3 bytes.
- El consumidor detecta el tamaño, decodifica y grafica igual que con JSON.
- Las curvas son idénticas en forma y rango a las de JSON: la compresión no perdió información (temperatura con 2 decimales, humedad entera, 8 direcciones).
- 15 muestras recibidas correctamente.



Evidencia: gráfica en vivo con payload compacto de 3 bytes

PREGUNTAS

Preguntas · Modelo Pub/Sub de Kafka

¿Qué ventajas y desventajas tiene el modelo Pub/Sub de Kafka?

- **Ventajas:** desacopla productores y consumidores (no se conocen ni tienen que estar activos a la vez); el log persistente permite releer mensajes desde un offset y recuperarse de caídas; escala horizontalmente con particiones y grupos de consumidores; alto throughput y orden garantizado dentro de cada partición; varios consumidores independientes pueden leer el mismo stream.
- **Desventajas:** la infraestructura es pesada para operar (brokers, replicación, ZooKeeper/KRaft); cada mensaje lleva overhead (headers, key, offsets, TCP) que pesa mucho con payloads diminutos; no hay orden global entre particiones; el cliente es demasiado grande para microcontroladores, por lo que se necesita un gateway; la entrega exactly-once requiere configuración adicional.

¿Para qué aplicaciones tiene sentido usar Kafka?

- Telemetría IoT agregada en un gateway o edge server, logs y métricas, event sourcing, pipelines de datos en tiempo real, CDC entre bases de datos y comunicación entre microservicios. No tiene sentido para RPC petición-respuesta de baja latencia ni para sistemas pequeños con pocos mensajes, donde MQTT o una cola simple bastan.

PREGUNTAS

Preguntas · Payload restringido

¿Qué complejidades introduce tener un payload restringido (pequeño)?

- Se pierde el formato autodescriptivo: emisor y receptor deben compartir exactamente el mismo esquema de bits, y cualquier cambio rompe la compatibilidad (no cabe un campo de versión).
- Hay que fijar rangos y precisión: valores fuera de rango se recortan y los floats se cuantizan.
- Más lógica y más riesgo de errores (desplazamientos, máscaras, endianness); la depuración es más difícil porque los mensajes ya no son legibles.
- No sobra espacio para metadatos (timestamp, id del sensor, checksum), que deben inferirse del canal.

¿Cómo podemos hacer que el valor de temperatura quepa en 14 bits?

- Convertirla a **punto fijo**: multiplicar por 100 y redondear a entero. El rango 0.00–110.00 se vuelve 0–11000, y $11000 < 2^{14} = 16384$, así que cabe en 14 bits sin pérdida para 2 decimales. Al decodificar se divide entre 100.

PREGUNTAS

Preguntas · Humedad flotante y restricciones más fuertes

¿Qué pasaría si la humedad también fuera float con un decimal? ¿Qué decisiones habría que tomar?

- 0.0–100.0 con un decimal son 1001 valores → requiere **10 bits**. Con 14 + 10 + 3 = 27 bits ya **no cabe** en 24.
- Opción A: bajar la temperatura a 1 decimal (0–1100 → 11 bits): 11 + 10 + 3 = **24 bits exactos**.
- Opción B: mantener la precisión y pasar a un payload de 4 bytes, si la red lo permite.
- Opción C: reducir el rango útil (p. ej. temperatura −10 a 60 °C) o cuantizar con pasos de 0.5. En todos los casos hay que decidir qué precisión es aceptable para la aplicación.

¿Qué parámetros o herramientas de Kafka ayudarían si las restricciones fueran aún más fuertes?

- `compression.type` (lz4, zstd, snappy, gzip) junto con `batch.size` y `linger.ms` para agrupar y comprimir varios mensajes por envío.
- Serializadores binarios (Avro / Protobuf) con **Schema Registry**: el esquema viaja aparte y el mensaje solo lleva datos y un id de esquema.
- Omitir la key o usar keys cortas; `acks` y `max.message.bytes` para controlar tráfico y tamaño. Kafka Streams o Connect en el edge para agregar lecturas antes de reenviarlas.

CIERRE

Conclusiones

- Se completaron las 4 fases: simulación de sensores, publicación a Kafka, consumo con gráfica en vivo y codificación compacta de 3 bytes.
- Kafka desacopla limpiamente al sensor del visualizador: productor y consumidor se ejecutan, detienen y reinician de forma independiente.
- Con punto fijo y empaquetado de bits la lectura pasa de 63 bytes en JSON a 3 bytes (≈ 95 % menos) sin perder la precisión requerida, verificado con 100 000 pruebas de ida y vuelta.
- El costo es un esquema rígido compartido: cualquier cambio de precisión o de campos (como humedad con decimales) obliga a rediseñar el layout de bits.
- En un despliegue real, los dispositivos LPWAN enviarían a un gateway, y este publicaría en Kafka, donde la compresión por lotes y los esquemas binarios reducen todavía más el overhead.